

בסוף התהליך לוקחים לכל אחד Z-1 מכל משאב, כלומר משאירים לו Z מכל משאב.

ארבעה משאבים, מכל אחד יש 1: שוקולד, עוגת טארט, סוכריות מנטה, עוגיות דלעת. שלושה שחקנים: הארי, רון והרמיוני.

קודדתי בפייתון פתרון לבעיה הבאה (בעזרתו האדיבה של צ'אט ג'יפיטי, מספיק משעמם כדי שאני לא אטרח להעתיק את השיחה, מחילה).

שוקולד	עוגה	סוכריות	עוגיות	
0	10	10	20	הארי
10	15	0	5	רון
20	5	10	15	הרמיוני

לפי ההרצה בפייתון, הארי מקבל את כל העוגיות, רון את כל העוגה, הרמיוני את כל השוקולד והארי והרמיוני מתחלקים בסוכריות חצי חצי.

זוה סך הכל,

$$(20+5)*15*(20+5) = 9,375$$

(יואו איזה מגניב זה השלים לי את הפתרון עכשיו!)

Z של הארי: $Y = 15*25 = 375$.

X זה (שוב הפעלה של פייתון, כבר חוזרת) שרון מקבל את העוגה ו-3/8 מהשוקולד, והרמיוני את השאר. שזה בעיקרון 18.75 לרון ו-37.5 להרמיוני, וסך הכל 703.125.

מה שאומר ש-Z של $375/703.125 = 8/15$.

Z של רון: $Y = 25*25 = 625$.

X זה שהארי מקבל את העוגה והעוגיות והרמיוני את השוקולד והסוכריות, הערך של שניהם זה שלושים אז

Z זה $25/36$.

Z של הרמיוני: Y כמו של הארי, 375.

X - הארי מקבל את העוגיות והסוכריות ורון את העוגה והשוקולד, כלומר הארי 30 ורון 25. סך הכל 750. לכן

Z של הרמיוני הוא $1/2$ שזה קצת מעניין וקצת עצוב, לדעתי.

סך הכל: הארי מקבל $25 \cdot \frac{8}{15} = 13 \frac{1}{3}$
רון מקבל $15 \cdot \frac{25}{36} = 10.4167$,
והרמיוני מקבלת $\frac{25}{2}$, שזה 12.5.

מעניין שהארי והרמיוני כבר לא מקבלים אותו דבר. בכללי האלגוריתם הזה אולי מגלה אמת, אבל
בהחלט קיצץ בחצי להארי והרמיוני, ולרון בשני שליש, מרגיש קצת מוגזם?

נניח ש $V_i(x)$ זה הערך ששחקן מצהיר על חלק מסוים.

נסתכל על שחקן מספר 1 בה"כ ונזכיר שיש לו אינטרס להצהיר את הערך האמיתי שלו.

נאמר ש- $V(x)$ זה הערך האמיתי של השחקן עבור חלק x כלשהו, ו- V_1 זה הערך שהוא מצהיר.

X_1 זה החלק שהשחקן קיבל בפועל בחלוקה היעילה-נאש בהתחלה. בסך הכל השחקן קיבל מבחינתו:

$$Z \cdot V(x_1) = \frac{\prod_{2 \leq i \leq n} V_i(x_i)}{\prod_{2 \leq i \leq n} V_i(x'_i)} \cdot V(x_1)$$

כשזו זה החלק ששחקן i קיבל, ו- x'_i זה החלק שהיה מקבל בלי השתתפותו של שחקן 1.

המכנה לא תלוי בכלל בהצהרתו של השחקן, לכן מבחינת השחקן הוא היה רוצה למקסם את

$$\prod_{2 \leq i \leq n} V_i(x_i) \cdot V(x_1)$$

$$\prod_{2 \leq i \leq n} V_i(x_i) = \prod_{2 \leq i \leq n} V_i(x_i) \cdot V_1(x_1)$$

כיוון שבהגדרה אנחנו ממקסמים את

$$\prod_{2 \leq i \leq n} V_i(x_i) \cdot V(x_1)$$

אם השחקן יצהיר על ערכו האמיתי, כלומר אם $V_1 = V$, הערך של

ימוקסם ממילא ולכן יש לשחקן אינטרס להצהיר על הערכים האמיתיים.

באופן אינטואיטיבי: לשחקן יש אינטרס שגם שאר השחקנים יקבלו כמה שיותר.

כל שורה בvaluations זה שחקן, כל טור זה משאב (אוכל!), משמאל לימין: עוגיות, סוכריות, עוגה ושוקולד.

```
>>> import cvxpy as cp
>>> import numpy as np
>>> valuations = np.array([
... [20,10,10,0],
... [5,0,15,10],
... [15,10,5,20]])
>>> x = cp.Variable((3,4))
>>> utilities = cp.sum(cp.multiply(valuations,x),axis=1)
>>> objective = cp.Maximize(cp.sum(cp.log(utilities)))
>>> constraints = [ x>=0, cp.sum(x,axis = 0)<=1]
>>> prob = cp.Problem(objective,constraints)
>>> prob.solve()
np.float64(9.14580179693349)
>>> print(np.round(x.value, 3))
[[1.  0.5 0.  0. ]
 [0.  0.  1.  0. ]
 [0.  0.5 0.  1. ]]
>>> print(np.round(utilities.value, 3))
[25.002 15.    24.998]
>>>
```

בחלק השני, כשחישבתי לכל שחקן את מכפלת התועלות של שאר השחקנים, בעיקרון שיניתי את הערכים למערך של פרמטרים, ואז שיניתי אותו כל פעם ופתרתי שוב, הנה הסוף של זה:

```
>>> prob.solve()
np.float64(6.555534679631421)
>>> print(np.round(x.value, 3))
[[0.  0.  1.  0.375]
 [1.  1.  0.  0.625]]
>>> val.value = np.array([[20,10,10,0],[15,10,5,20]])
>>> prob.solve()
np.float64(6.802394733833287)
>>> print(np.round(x.value, 3))
[[1. 0. 1. 0.]
 [0. 1. 0. 1.]]
>>> val.value = np.array([[20,10,10,0],[5,0,15,10]])
>>> prob.solve()
np.float64(6.620073147456027)
>>> print(np.round(x.value, 3))
[[1. 1. 0. 0.]
 [0. 0. 1. 1.]]
>>>
```